PROJECT CONTEXT · FOR RESUMING WORK

EchoPath on Jetson Nano

State of play, decisions, measurements and open issues

Tanna TechBiz **Eagle 101** + NVIDIA Jetson Nano production module

BOARD STATUS	WORKSTATION PATH	DEVICE PATH	TOP RISK
Not yet flashed	Working, verified	Written, untested	chair recall 0.30

0 · How to use this document

This is a handover, not a transcript. It records **what was decided and why, what was measured, and what is still open** — the things that would otherwise have to be rediscovered. Where an earlier conclusion turned out to be wrong, the correction is recorded in §7 rather than quietly fixed, because the wrong version is the intuitive one and will be re-proposed otherwise.

EchoPath is a wearable assistive-navigation system for visually impaired users in Indian academic buildings. A camera feeds YOLOv8n (17–19 indoor classes) and MiDaS Small (monocular depth); detections are fused into *object / proximity / direction* and spoken aloud. The published model is Training v3, mAP@50 = 0.869. Development is on an RTX 4500 Ada workstation; deployment target is the Jetson Nano board above.

1 · Hardware — confirmed from the retail unit

Item	Value	Consequence
Module	Jetson Nano **production** (P3448-0020), Tegra X1, Maxwell	128 CUDA cores, ~472 GFLOPS FP16. EOL — JetPack 4.6.x is terminal.
RAM	4 GB LPDDR4, **shared CPU/GPU**	No separate VRAM. TensorRT engine builds OOM without mitigation.
Storage	**16 GB eMMC** (soldered) + microSD slot	Full JetPack ≈14 GB. Rootfs must go on the microSD.
Compute capability	**SM 5.3**	**No INT8** — no DP4A instruction. FP16 is the floor.
Camera	2× MIPI CSI-2, 2-lane, 15-pin	Use `nvarguscamerasrc`; hardware ISP, no CPU cost.
USB	4× USB 3.0 A; 1× Type-B *data only*	Type-B is the recovery/flash port.
Audio	**None** — no analog codec	USB audio dongle is a mandatory BOM item.
Power / cooling	5 V / 4 A barrel; fan included	Budget ~10 W sustained under MAXN.

HARDWARE ON HAND

- **128 GB SanDisk Ultra microSDXC UHS-I A1** — adequate. A2's advantage needs host features the Nano lacks, so A1 performs comparably here. Verify authenticity with `f3` or `h2testw` before trusting it.
- **HP 64 GB USB 3.2 pendrive** — *not* suitable as rootfs (poor random write IOPS, wears out). Good for file transfer, holding the 13 test videos, and imaging a working card.
- Test footage was shot on an **iPhone 17**, portrait 1080×1920.

STILL TO ACQUIRE

USB audio dongle · IMX219 CSI camera · **x86_64 host running Ubuntu 18.04/20.04** (bare metal or dual boot, not a VM) · Tanna TechBiz BSP · 5 V/4 A capable battery pack.

> **Check before buying a Linux host**
> Power the board on with HDMI attached first. Some vendors ship the module **pre-flashed**. If it boots, you may be able to keep the OS on eMMC and relocate `/home`, swap and engines to the microSD via `fstab` — avoiding the flash procedure entirely. If it does not boot, or boots without CSI camera support, you need the x86 host. Ask Tanna directly whether modules ship pre-flashed and with which JetPack configuration.

---

2 · The software ceiling

Component	Workstation	Jetson ceiling	Verdict
Python	3.11	**3.6**	hard floor
PyTorch	2.4.1+cu118	1.10.0 (last cp36 aarch64)	cannot install
Ultralytics	8.4.x	requires ≥3.8	cannot install
NumPy	1.26.4	1.19.5	inherited from JetPack
OpenCV	4.11 (pip)	4.1.1 (JetPack, w/ GStreamer)	**never pip install**
CUDA / TensorRT	11.8 / —	10.2 / 8.2	use TensorRT directly

Therefore the deployment strategy is: export to ONNX on the workstation, build TensorRT engines *on the board*, run inference through the TensorRT runtime + PyCUDA, with no PyTorch and no Ultralytics on the device.

> **Never pip install opencv-python on the device**
> It shadows JetPack's build, which is the only one compiled with GStreamer. The `nvarguscamerasrc` CSI pipeline then silently stops opening, and the failure looks like a camera fault with nothing pointing back at the pip install.

3 · Repository map

3.1 The canonical file

`Jetson_Implementation/EchoPath_Jetson.ipynb` — this is the file to work on. It is a copy of the user's own `with_TTS (2)` notebook with the alert layer rewritten and a TensorRT backend added. Detection, depth and fusion logic are *unchanged* from the original.

Path	Status	What it is
`Jetson_Implementation/`	CANONICAL	Self-contained deployment folder — see below
`midas_small/EchoPath_Pipeline_MiDaS_v2_with_TTS (2).ipynb`	REFERENCE	The user's original working pipeline. Produced the paper's outputs. Untouched.
`EchoPath_Jetson_Pipeline.ipynb`	PARALLEL	A from-scratch restructure. Has the ONNX export, ablation and geometric work that *Jetson_Implementation lacks*.
`echopath_run.py` / `echopath_device.py`	PARALLEL	Generated from the above by scripts in the scratchpad.
`TTS_integrated_model.ipynb`, `echopath_test.ipynb`	RETIRE	Superseded / scratch.
`onnx/` (repo root)	READY	5 ONNX files. **Not yet copied into Jetson_Implementation.**

> **Two parallel lines of work exist — this is the main source of confusion**
> `EchoPath_Jetson_Pipeline.ipynb` (plus its two generated `.py` files) was built first and contains the ONNX export, the resolution ablation, the geometric ranging arm and four Jetson performance optimisations. The user then asked to work from their own notebook instead, producing `Jetson_Implementation/`. **Those four optimisations were never ported across** — see §6. Decide whether to port them or to consolidate on one line.

3.2 Inside `Jetson_Implementation/`

File	Purpose
`EchoPath_Jetson.ipynb`	25 cells. Dual backend: cell 1 detects the board and sets `BACKEND`.
`env/`	Isolated conda prefix env, Python 3.11, 6.4 GB. Activate by *path*.
`check_jetson_compat.py`	Automated gate: Python 3.6 syntax + device-allowed imports over the device cells.
`environment.yml` / `requirements-workstation.txt`	Workstation spec.
`requirements-jetson.txt`	Device spec — essentially just `pycuda`; everything else is inherited from JetPack.
`SETUP.md`	Setup for both machines, cell map, traps.
`assets/audio/`	121 pre-rendered alert WAVs, 12.5 MB.
`output/`	13 annotated + 13 voiced videos.

CELL MAP — WHICH CELLS RUN WHERE

```
cells 1–4   probe, config, models, fusion + AlertPolicy      BOTH
cell  5–8   calibration plots, rotation, batch video, summary  workstation only
cell  9     vocabulary render + TTSSpeaker                     BOTH
cell 10–11  narration, bake voice into mp4                     workstation only
cell 12     live CSI camera runtime                            DEVICE
```

Environment activation is by path, not name:

```
conda activate E:\EchoPath\Jetson_Implementation\env
```

`conda env create -f environment.yml` fails on this machine — Anaconda ToS on the `defaults` channels. Use `conda create -p <path> --override-channels -c conda-forge ...` then pip.

---

4 · Measured results

4.1 Resolution ablation — 2,780 validation images

imgsz	rel. pixels	mAP@50	mAP@50-95
640	1.00	0.8691	0.6983
512	0.64	**0.8775**	0.7084
416	0.42	0.8705	0.7057
320	0.25	0.8483	0.6871

640 reproduces the paper's 0.869 exactly, confirming the right model is under test.

> **But this table is misleading, and acting on it was a mistake**
> Measured on validation *images*. On the actual *videos* (1080×1920 portrait), letterboxing to 416 leaves ~234 px of content and loses detections badly: **Classroom 163 → 78 (−52%)**, lab 128 → 104, first_floor 121 → 113. End-to-end alerts across 5 videos fell 51 → 43. **Decision: keep `imgsz = 640`.** Re-measure on real IMX219 landscape frames before trading resolution for Jetson latency.

4.2 Alert policy — 13 videos, 7,281 frames

Metric	Value	Note
Alerts, MiDaS mode	**73**	Paper Fig. 7 counted 261 on the same footage — 3.6× reduction
Alert rate	18.0 / minute	~one every 3.3 s
Most-alerted class	chair (29 of 73)	40% of all alerts, *despite* 0.30 recall
Proximity split	close 33 / medium 40	
Phrase durations	1.29–3.18 s, median 2.36 s	Measured from the rendered WAVs
MIN_GAP_S / cooldown	**3.5 s**	Must exceed the longest phrase

4.3 Depth: MiDaS vs geometric ranging

Metric (11 approach tracks)	Geometric	MiDaS
Mean |ρ| (Spearman, distance vs time)	**0.936**	0.718
Correct direction of travel	**11 / 11**	8 / 11

Metric (11 approach tracks)	Geometric	MiDaS
Ever reaches `close`	—	**3 / 11**
Per-frame cost (RTX 4500)	0 ms	**45 ms** (YOLO is 22 ms)
Throughput	45 fps	34 fps

MiDaS gets the direction backwards on 3 of 11 approaches — it reports a chair receding while the camera walks into it. And **8 of 11 actively-approached obstacles never get classified** `close`, because the P75 percentile threshold is consumed by the floor in front of the camera.

MiDaS also costs *twice* what YOLO does. On Maxwell that ratio worsens.

> **Not yet decided**
> Geometric tracks approach better and is free, but its absolute scale is uncalibrated — `f_px` is estimated (±7% given the known iPhone camera), and `Y_HORIZON` assumes a level camera (±25%). Class height priors add ±20–30% and are irreducible. One tape-measure session removes two of the three. The geometric arm currently lives only in `EchoPath_Jetson_Pipeline.ipynb`, not in the canonical folder.

4.4 Other verified facts

- **MiDaS ONNX parity = 1.0000** cosine similarity against PyTorch on 3 frames. Export is lossless.
- ONNX output shape `[1, 23, 3549]` → 23 − 4 = **19 classes**.
- Dataset (`dataset_merged_v3`): 41,782 instances. Dominant class is `fire_extinguisher` at 13,107; `handbag` has 6; `backpack` has 1,238 train but **0 validation**. Box aspect ratios confirm labels are self-consistent (`column` h/w = 7.05, `table` = 0.67).
- Annotation + video writing costs ~⅓ of throughput (34 → 21 fps on RTX). Disabled on device.

5 · Decisions and rationale

Decision	Why
FP16, never INT8	Maxwell SM 5.3 has no DP4A. `--int8` warns and *silently falls back* — the engine looks quantised and performs like FP16. Check `--verbose` for FP32 fallbacks too.
`imgsz = 640`	416 loses up to 52% of detections on portrait video. The image-based ablation did not show this.
Pre-rendered WAV vocabulary	Alert grammar is a closed set (121 phrases). Synthesis on 4× A57 is avoidable; and a file played by a subprocess can be *killed*, which is what makes urgent alerts able to interrupt.
CSI camera, not USB	USB webcams send MJPEG; decoding it costs 30–45% of one A57 core continuously. CSI goes through the Tegra ISP at no CPU cost.
Rootfs on the 128 GB microSD	16 GB eMMC cannot hold JetPack (~14 GB) + 8 GB swap + engines.
`CLASS_NAMES` **has 19 entries**	The `.plan` carries weights but not names. Using the paper's 17 shifts every index above 10 — `wet_floor_sign` would report as `monitor`, which is silenced, so a wet-floor hazard would produce **no alert**. Asserted at engine load.
Natural phrasing retained	User's call. Costs ~1 s per alert vs terse form, hence the 3.5 s gap. Revisit later.
Conda prefix env inside the folder	User wants the folder isolated and portable.

5.1 What the three alert-policy fixes were

Gap	Was	Now
No priority	`queue.Queue()` unbounded FIFO — a `close` alert queued behind a playing `medium` one	Depth-1 priority slot; higher priority replaces what is waiting *and* cuts off what is playing
Four cooldowns	1.5 s and 5.0 s in cell 9, 2.0 s in cell 11, paper says 2.5 s. Repeat cooldown keyed on the *phrase string*, so `medium` → `close` bypassed it	One `AlertPolicy`, keyed on `(label, direction)`
Backlog	Emitted every 1.5 s while phrases take up to 3.2 s — unbounded queue meant audio drifted seconds behind reality	`MIN_GAP_S = 3.5` ≥ longest phrase; depth-1 slot makes overrun *drop*, not lag

Verified across 13 videos: `POLICY HELD`, min gap 3.00 s, cooldown 3.07 s (at the earlier 3.0 s setting), zero violations. Audio test: `spoken=2, preempted=1, fallback=0`.

6 · Open issues, prioritised

#	Issue	Blocked by	Detail
1	`chair` **recall 0.30**	NOTHING	Highest-value item overall. Chairs are the commonest trip hazard and 40% of all alerts. v2 scored 1.00 on the same class — prime suspect is v3's mixup / copy-paste augmentation. One ablation retrain would settle it.
2	**Four Jetson optimisations missing** from the canonical notebook	NOTHING	(a) both depth backends still upsample 256^2 to full frame; (b) `np.percentile` still called twice *inside* the per-detection loop; (c) no threading in the live loop; (d) `show=True` re-runs depth every frame, ignoring `depth_every_n`. ~40 lines. This is the difference between ~5 FPS and something usable.

#	Issue	Blocked by	Detail
3	Folder not self-contained	NOTHING	`onnx/` not copied in; no ONNX export cell in the canonical notebook, so artefacts cannot be regenerated after a retrain.
4	TensorRT backend never executed	PARTLY	Can be de-risked now by dry-running the decode against ONNX with onnxruntime — validates output tensor layout (the `p.shape[0] < p.shape[1]` transpose heuristic) and NMS. Rest needs the board.
5	Geometric arm not in canonical folder	NOTHING	Exists in the parallel pipeline. Decide whether to port.
6	Depth-mode decision	CAMERA	Needs `f_px`, `CAM_HEIGHT_M`, `Y_HORIZON` measured with the IMX219.
7	Resolution vs latency	BOARD	`trtexec` latency at each imgsz vs detection counts on real camera frames.
8	Flash, engines, camera, audio	BOARD	See §1 and SETUP.md.
9	Paper amendments	NOTHING	Table I quotes RTX 4500 numbers (29 FPS) while arguing Jetson feasibility. Also 17-vs-19 class inconsistency, and per-class metrics over as few as 6 instances.

7 · Corrections — claims that were wrong

Recorded because each wrong version is the intuitive one and will otherwise be re-proposed.

Claimed	Correct position
Use INT8 on the Jetson	**Impossible.** Maxwell SM 5.3 lacks DP4A. FP16 only. This removed a whole optimisation lever and made resolution the sole latency dial.
"416 px is free — same mAP as 640"	**False for video.** True on the validation image set; on 1080×1920 portrait footage it loses up to 52% of detections. Measure on the input distribution you will actually use.
Geometric distance is "absolute"	Metric *in kind*, but uncalibrated. Also: I blamed `f_px` (±50%) when the real error was `Y_HORIZON = 0`, worth **2–25×** on the ground-plane cue. `f_px` is only ±7% given the known camera.
"Use an A2 card, A1 is markedly worse"	Overstated. A2's advantage needs host command-queuing the Nano lacks. **A1 is fine here.**
`backpack` / `handbag` have zero instances	**Wrong.** backpack has 1,238 train (0 valid); handbag has 6. Was inferred from report figures rather than the label files.
"Pre-rendered WAVs, zero synthesis on device"	Was true of the *parallel* pipeline, not the canonical notebook, for a long stretch. Now implemented in both.
espeak burns 1–2 s of CPU per phrase	1–2 s is the *speaking* duration. espeak synthesis is fairly cheap. The real arguments for pre-rendering are interruptibility and device routing.
The 17-vs-19 class mismatch is a live bug	It is a **future hazard**, not a present defect. The user's notebook reads `yolo_model.names` and cannot be wrong. It only bites when a name list is hand-written for the TensorRT path.

7.1 Process lessons

- **Never `str.replace()` without asserting it matched.** A silent no-op left the old alert dedup in place and produced 67 alerts for an 18.9 s video. Use line ranges or assert the hit count.
- **Never pipe a long background run through `tail`.** It buffers until exit; a job deadlocked for 16 minutes with no visible output.
- **One writer at a time.** Two concurrent pip installs into the same env caused `WinError 32`; two processes writing the same JSONL produced empty logs.
- **Windows multiprocessing needs `if __name__ == "__main__":`** — DataLoader workers re-import the driver script and deadlock without it.
- **Prefer self-checking code over prose.** The load-time class assertion, the ONNX parity gate, the alert-spacing audit and `check_jetson_compat.py` each caught real defects; comments describing the same rules did not.

8 · Next actions

NO HARDWARE NEEDED — IN PRIORITY ORDER

1. **Port the four optimisations** into `Jetson_Implementation` (issue 2). Depth at native 256^2, hoist the percentiles, thread the live loop, drop annotation from the runtime path.
2. **The `chair` ablation retrain** (issue 1) — disable mixup, compare per-class against v2/v3.
3. **Make the folder self-contained** — copy `onnx/` in, add an export cell.
4. **Dry-run the TRT decode** against ONNX with onnxruntime to validate tensor layout.
5. Decide whether to consolidate the two parallel pipelines.

HARDWARE-GATED

6. Power on with HDMI — is it pre-flashed? That answer decides whether an x86 Ubuntu host is needed at all.
7. Flash with Tanna's BSP, rootfs on the microSD; headless boot; 8 GB swap.

8. Build engines: `trtexec --fp16 --verbose --workspace=512`, one at a time.
9. Verify camera and audio *independently* before running the pipeline.
10. Calibrate `f_px`, `CAM_HEIGHT_M`, `Y_HORIZON`; then settle the depth-mode question.
11. Run the 13 videos on-device and diff the alert logs against the workstation baseline.

Appendix · Key commands

```
# workstation env (prefix, inside the folder)
conda create -p E:\EchoPath\Jetson_Implementation\env -y --override-channels \
    -c conda-forge python=3.11 pip ipykernel jupyter
env\python.exe -m pip install torch==2.4.1+cu118 torchvision==0.19.1+cu118 \
    --index-url https://download.pytorch.org/whl/cu118
env\python.exe -m pip install -r requirements-workstation.txt
python check_jetson_compat.py            # run before every commit

# device: environment
python3 -m venv --system-site-packages ~/echopath-env   # inherits tensorrt + cv2
pip install pycuda                                       # the only real install
sudo apt install -y espeak libespeak1 alsa-utils

# device: prepare for the engine build
sudo systemctl set-default multi-user.target             # frees ~1 GB of 4 GB
sudo fallocate -l 8G /swapfile && sudo chmod 600 /swapfile
sudo mkswap /swapfile && sudo swapon /swapfile
sudo nvpmodel -m 0 && sudo jetson_clocks

# device: build engines (one at a time, FP16 only)
/usr/src/tensorrt/bin/trtexec --onnx=onnx/yolov8n_640.onnx --fp16 --verbose \
    --workspace=512 --saveEngine=engines/yolo_640_fp16.plan

# device: verify hardware independently BEFORE running the pipeline
gst-launch-1.0 nvarguscamerasrc num-buffers=300 ! \
  'video/x-raw(memory:NVMM),width=1280,height=720,framerate=30/1' ! nvvidconv ! fakesink -v
aplay -l                                  # find the USB dongle, set ALSA_DEVICE
dmesg | grep -i "killed process"          # if a build dies silently, look here
```
